

SATE AI: A Fault Injection and Reliability Engineering Framework for On-Device AI Models in Mobile Applications

Muhammad Assad Ullah

University of Engineering and Technology Taxila / Independent Researcher

asadullahaj602@gmail.com

October 9, 2026

Abstract

On-device artificial intelligence deployment in mobile applications confronts machine learning models with volatile hardware environments: unpredictable memory pressure, dynamic thermal constraints, and corrupted sensor inputs. Cloud-hosted systems enjoy active cooling, stable resource budgets, and established chaos engineering toolchains; mobile edge AI has none of these guarantees. Unhandled model failures manifest as silent process eviction, uncaught runtime exceptions, or gradual output quality degradation that users notice before developers do.

We present SATE AI, an open-source fault injection and reliability engineering framework designed for on-device machine learning models in mobile and desktop cross-platform applications built with Dart and Flutter. SATE AI provides a unified architecture comprising 11 domain-specific fault injectors, covering process memory pressure, thermal throttling simulation, malformed tensor inputs, quantization drift, data corruption, network latency drops, model swaps, version mismatches, GPU memory pressure, and confidence threshold degradation—paired with 8 model runtime adapters spanning TensorFlow Lite, ONNX Runtime, llama.cpp/flama, Apple CoreML, MediaPipe, Google ML Kit, MockAdapter, and a custom extensibility interface.

Framework correctness is validated through a 246-case automated test suite covering unit, integration, and stress scenarios across all 11 fault injectors and all 8 model adapters. This paper describes the framework design, architecture contracts, and test-suite validation. A follow-up empirical study applying these injectors to physical Android hardware and measuring runtime degradation via EdgePulse telemetry [9] is planned as a companion contribution.

SATE AI is open-source software, available at https://pub.dev/packages/sate_ai and https://github.com/assassinaj602/sate_ai, under the MIT license.

1 Introduction

Quantized large language models (LLMs) and compact computer vision architectures are increasingly embedded directly into mobile applications. Runtimes such as TensorFlow Lite [1], ONNX Runtime [4], and llama.cpp [3,6] make local inference practical on consumer hardware, eliminating cloud API costs, reducing round-trip latency, and keeping user data on-device.

The shift from cloud inference to mobile client execution, however, transfers reliability engineering responsibility onto application developers who may have little visibility into hardware behavior below the OS abstraction layer. Mobile SoCs operate under energy budgets, junction temperature ceilings, and RAM limits that have no equivalent in a data center rack.

1.1 Hardware-Level Failure Modes

Three failure vectors dominate on-device AI reliability:

Thermal dissipation limits. Sustained dense matrix operations elevate SoC junction temperatures. When junction temperature approaches vendor-defined limits, operating system kernels enforce Dynamic Voltage and Frequency Scaling (DVFS) [8], reducing clock speeds or disabling performance cores. The result is a latency spike that appears intermittently and correlates with device skin temperature rather than with input characteristics—making it difficult to reproduce in a standard unit test.

System memory eviction. Working set expansion during LLM prompt processing or key-value (KV) cache generation can exhaust available RAM. Android’s Low Memory Killer (LMK) terminates background—and occasionally foreground—processes to reclaim pages; iOS’s Jetsam daemon does the same under memory pressure. A model that passes all functional tests may be silently killed under realistic multi-app conditions.

Input and quantization volatility. Sensor noise, camera artifact corruption, or low-precision weight quantization (4-bit GGUF [3,7], INT8 TFLite [1,2]) can trigger numerical instability, unexpected confidence drops, or runtime exceptions that appear only when the model receives data that differs statistically from its development test set.

1.2 The Reliability Testing Gap

Chaos engineering frameworks—Chaos Mesh, Gremlin, and related tools [8]—inject infrastructure-level faults to probe cloud service resilience. They are well-suited to distributed systems with explicit network boundaries. Mobile on-device inference has no such boundary: the model executes inside the application process, interacting with device hardware through OS abstractions that these tools cannot reach.

Standard mobile ML testing fills none of this gap. A typical unit test instantiates a model wrapper, feeds clean synthetic tensors, and asserts on pre-defined output arrays. It cannot answer whether the application gracefully handles a memory pressure event mid-inference, whether a 4-bit quantized model’s confidence distribution holds under tensor corruption, or whether the application UI falls back correctly when model latency triples due to thermal throttling.

1.3 Contributions

This paper contributes:

1. **SATE AI Architecture.** An open-source, extensible fault injection and reliability engine establishing unified `ModelAdapter` and `FaultInjector` contracts for cross-platform mobile applications.
2. **Fault Injector and Adapter Suite.** Eleven domain-specific fault injectors and eight model runtime adapters, enabling systematic stress testing across TFLite [1], ONNX Runtime [4], llama.cpp/flama [3,6], CoreML [7], and MediaPipe [6].
3. **Baseline and Regression Engine.** A baseline management subsystem (`BaselineManager`, `QuantizationAnalyzer`, `ReportComparator`) that records golden execution profiles, calculates metric deviations, and exports structured HTML/JSON benchmark reports.

4. **Validated Open Artifact.** A fully tested open-source implementation with 246 passing test cases released under the MIT license on pub.dev. Physical device experiments are left to a companion empirical study [9].
-

2 Background and Related Work

2.1 On-Device Machine Learning Runtimes

Modern mobile applications deploy deep neural network inference through specialized runtime engines optimized for the memory and power constraints of mobile SoCs.

TensorFlow Lite (TFLite) [1] is Google’s execution environment for quantized neural networks on mobile and embedded hardware. TFLite supports integer quantization (INT8, FP16) [2] and delegates computation to hardware-specific accelerators via the Neural Network API (NNAPI) on Android and the GPU delegate on iOS. Its flat buffer model format minimizes memory allocation during model loading.

ONNX Runtime [4] is a cross-platform inference engine maintained by Microsoft and the Linux Foundation. It accepts models in the Open Neural Network Exchange format and supports a plugin architecture of execution providers—CPU, CUDA, DirectML, CoreML—that makes it suitable for cross-platform deployment without retraining.

llama.cpp / flama [3,6] provides an optimized C/C++ implementation of LLM inference on ARM CPU architectures. It targets 4-bit quantized GGUF model files [7] and implements hand-tuned SIMD matrix multiplication kernels that make billion-parameter model inference tractable on consumer mobile hardware without GPU acceleration.

CoreML and MediaPipe [6,7] offer platform-specific neural engine invocation on Apple silicon (CoreML) and cross-platform vision and multimodal pipeline processing (MediaPipe), respectively.

2.2 Fault Injection Methodologies

Fault injection as a technique for evaluating system robustness under adverse conditions has a long history in hardware and safety-critical systems engineering [5].

Hardware and micro-fault injection. Low-level techniques such as MicroFI [5] or pin-level voltage glitching inject faults directly into microprocessor instruction pipelines at silicon validation time. These approaches are indispensable for certifying processor designs, but they require specialized lab hardware, are destructive at high intensities, and cannot be integrated into a mobile software CI/CD pipeline. They also operate below the abstraction level at which application developers can observe or mitigate failures.

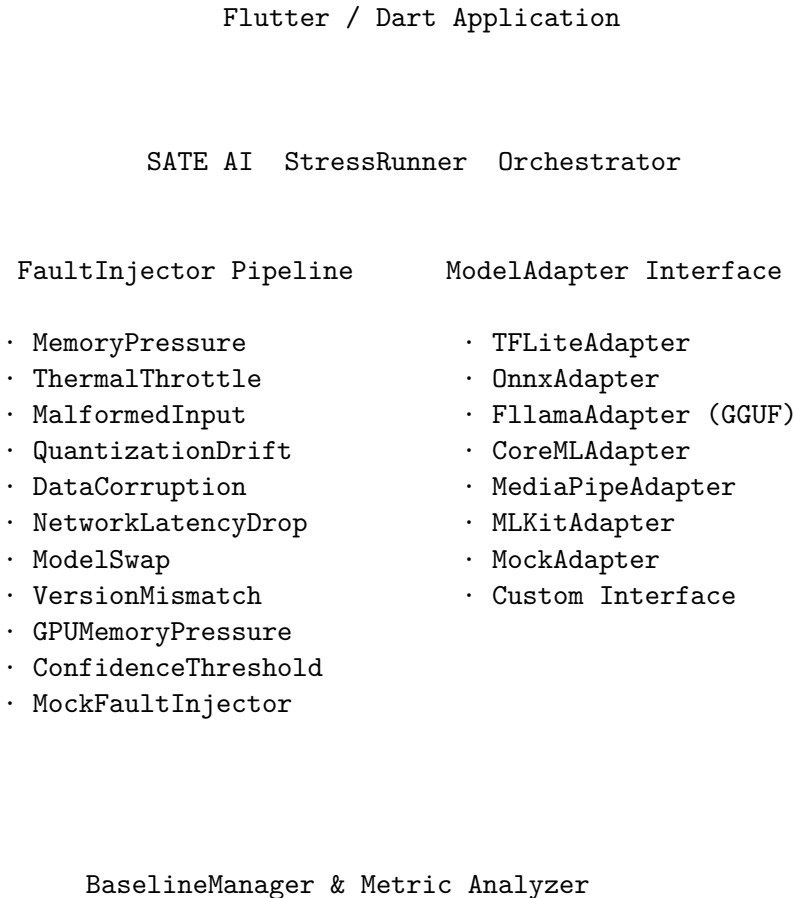
Network and service chaos engineering. Chaos Mesh, Gremlin, and Netflix’s Chaos Monkey [8] inject HTTP latency, packet loss, and service outages to test distributed system resilience. These tools assume a network boundary between components. On-device AI models execute within the application process; there is no network path to instrument. Chaos engineering tools provide no mechanism to stress a model’s memory allocations, tensor execution graph, or quantized weight layout.

Software fault simulation for ML. Work on neural network robustness evaluation [2] has examined how quantization affects model accuracy under distribution shift, and post-training quantization studies [3] characterize the precision-accuracy tradeoff for 4-bit quantized LLMs. Neither addresses runtime fault injection at the application layer during live inference. SATE AI occupies this gap: software-level fault injection at the model runtime boundary, within the application process, without requiring physical hardware instrumentation or network intermediaries.

The closest related work is the broader category of mutation testing and property-based testing in software engineering, where inputs are systematically corrupted to expose edge cases in application logic. SATE AI extends this principle to the on-device ML runtime boundary, where the “inputs” include not only tensor data but also the hardware resource environment in which inference executes.

3 System Architecture

SATE AI is built around a decoupled, layered architecture that separates model interaction, fault injection, execution orchestration, and reporting into independent subsystems connected through well-defined abstract contracts.



- Golden Baseline Recording (N-iteration warmup runs)
- Metric Deviation Bounds (ΔL , ΔC , ΔM thresholds)
- QuantizationAnalyzer (Precision degradation scoring)
- ReportComparator (Regression flagging engine)

Report Database & Export Backends

- JSON Report (structured machine-readable output)
- HTML Dashboard (Chart.js interactive visualizations)
- SQLite Storage (ReportDatabase persistence layer)

Figure 1. SATE AI layered architecture. Arrows represent data flow from the host application, through the orchestration layer, to the fault injection and adapter pipelines, and down to the metric analysis and reporting backends.

3.1 Metric Deviation Formalism

The baseline and regression engine quantifies three primary deviation metrics. Let B_x denote the golden baseline value of metric x and F_x the value observed under a fault scenario.

Latency delta measures the absolute change in per-inference wall-clock time:

$$\Delta L = L_{\text{fault}} - L_{\text{baseline}}$$

where L_{fault} and L_{baseline} are mean inference latencies (in milliseconds) over N measured iterations. A positive ΔL indicates degradation; SATE AI flags runs where $\Delta L/L_{\text{baseline}} > \tau_L$ for a user-configurable threshold τ_L (default: 0.15, i.e., 15%).

Output confidence drop captures normalized degradation in model output probability scores:

$$\Delta C = \frac{C_{\text{baseline}} - C_{\text{fault}}}{C_{\text{baseline}}}$$

where $C \in [0, 1]$ is the maximum softmax output probability (for classification models) or a normalized generation quality score. $\Delta C = 0$ indicates no confidence degradation; $\Delta C = 1$ indicates complete output collapse.

Peak memory overhead measures the difference between peak resident set size during inference and idle baseline consumption:

$$\Delta M = M_{\text{peak}} - M_{\text{idle}}$$

expressed in megabytes. For the **MemoryPressureInjector**, this quantity additionally accounts for the injector’s own allocation budget, which is tracked separately to isolate model-induced memory growth from injector overhead.

These three metrics form the basis for all regression detection and report generation in SATE AI.

3.2 ModelAdapter Contract

To support heterogeneous execution engines without coupling application code to runtime-specific APIs, SATE AI defines the `ModelAdapter` abstract contract:

```
abstract class ModelAdapter {
    Future<void> initialize();
    Future<ModelOutput> predict(ModelInput input);
    Future<void> dispose();
    bool get isInitialized;
    ModelMetadata get metadata;
}
```

Concrete implementations wrap target runtimes: `TFLiteAdapter` manages TensorFlow Lite interpreter delegates [1]; `OnnxAdapter` manages ONNX Runtime session lifecycle [4]; `FllamaAdapter` binds to the llama.cpp GGUF inference pipeline [3]; `CoreMLAdapter` invokes Apple Neural Engine compilation and execution [7]; `MockAdapter` provides a deterministic stub for unit testing without requiring a real model file.

The adapter contract deliberately excludes runtime-specific initialization parameters (delegate configuration, thread pool sizes, model paths). These are passed to each concrete adapter’s constructor and are not part of the shared interface. This keeps the `StressRunner` and `FaultInjector` layers decoupled from any particular runtime’s initialization conventions.

3.3 FaultInjector Contract

All fault injectors implement the `FaultInjector` base class:

```
abstract class FaultInjector {
    final FaultType type;
    final double severity; // Normalized severity parameter [0.0, 1.0]

    Future<void> prepare();
    Future<ModelInput> injectInput(ModelInput input);
    Future<ModelOutput> injectOutput(ModelOutput output);
    Future<void> cleanup();
}
```

The `severity` parameter is a normalized scalar $s \in [0.0, 1.0]$ that each injector maps to its own internal parameter range. At $s = 0.0$ the injector is a no-op (used for baseline runs without code path changes); at $s = 1.0$ it applies maximum stress within safe operating bounds.

The lifecycle methods `prepare()` and `cleanup()` bracket each fault scenario, ensuring that allocated resources (memory buffers, delay timers, corruption state) are fully released before the next scenario begins. This prevents fault state from leaking across test cases, which is verified explicitly by the unit test suite.

3.3.1 Built-in Fault Injectors

The 11 built-in fault injectors cover distinct failure vectors across the hardware–software stack:

1. **MemoryPressureInjector.** Allocates controlled native and heap memory buffers between 50 MB and 1.5 GB (scaled by s) to simulate concurrent background application memory pressure and trigger Android LMK or iOS Jetsam memory boundary conditions.
2. **ThermalThrottleInjector.** Introduces deterministic latency scaling multipliers between $1.2\times$ and $3.0\times$ (scaled by s) and interleaved execution pauses to simulate SoC DVFS clock gating [8] without requiring physical thermal stress.
3. **MalformedInputInjector.** Corrupts input tensors by injecting NaN, positive or negative Infinity, zero-padding, or out-of-bounds dimensional shapes. This tests tensor validation robustness and exception handling in adapter implementations.
4. **QuantizationDriftInjector.** Applies noise perturbation matrices to input data and output weight scores to simulate precision loss from low-bit quantization (FP32 to INT4 conversion artifacts [2,7]).
5. **DataCorruptionInjector.** Applies bit-flip operations to raw byte arrays or image pixel buffers to evaluate model resilience against sensor noise and storage corruption.
6. **GpuMemoryPressureInjector.** Simulates GPU texture buffer allocation saturation on platform GPU delegates by exhausting available OpenGL ES or Vulkan buffer pools within the application sandbox.
7. **LatencyInjector.** Adds configurable execution delay distributions—fixed delay, uniform jitter, or Gaussian jitter—to simulate variable hardware scheduling latency without modifying model weights or inputs.
8. **ModelSwapInjector.** Replaces the target model’s weights mid-session to test runtime fallback handling, version compatibility guards, and application state recovery after a model reload.
9. **ModelVersionMismatchInjector.** Passes incompatible schema configurations (mismatched input shape, unsupported opset version, incompatible metadata) to test version guardrail enforcement in adapter implementations.
10. **NetworkLatencyDropInjector.** Simulates transport latency for hybrid edge–cloud fallback pipelines, where on-device inference failure triggers a cloud API fallback with its own latency and connectivity constraints.
11. **ConfidenceThresholdInjector.** Artificially degrades the model’s output probability distribution—shifting logit values to reduce maximum confidence scores—to test application-layer confidence threshold logic and UI fallback behavior.

3.4 StressRunner Orchestration Engine

The **StressRunner** orchestrates fault scenarios in three phases:

Phase 1: Baseline recording. The runner executes N unfaulted inference iterations (default: $N = 30$) through the target **ModelAdapter** to establish the golden performance profile: mean latency L_{baseline} , peak memory M_{baseline} , and output confidence C_{baseline} .

Phase 2: Fault scenario execution. For each configured **FaultInjector**, the runner calls `prepare()`, then executes N inference iterations with the injector active (modifying inputs via

`injectInput()` before prediction and outputs via `injectOutput()` after prediction), then calls `cleanup()`. Metrics are collected per-iteration and aggregated.

Phase 3: Deviation analysis and reporting. The runner computes ΔL , ΔC , and ΔM for each fault scenario, compares against configured thresholds, flags regressions, and serializes results through the configured report backend (JSON, HTML, or SQLite).

3.5 BaselineManager and QuantizationAnalyzer

BaselineManager stores golden baseline records in a local SQLite database and provides a **ReportComparator** that evaluates whether a new run’s deviation metrics fall within acceptable bounds. It supports time-series baseline comparison, enabling regression tracking across application versions.

QuantizationAnalyzer evaluates precision degradation along the quantization chain $\text{FP32} \rightarrow \text{FP16} \rightarrow \text{INT8} \rightarrow \text{INT4}$. For each precision level, it computes a normalized degradation score incorporating both accuracy loss (relative to FP32 output) and latency change, providing a single scalar that teams can use to compare quantization strategies without running a full benchmark suite.

4 Verification and Test Suite Evaluation

The SATE AI test suite comprises 246 automated verification specifications organized in the `test/` directory of the repository. The suite validates framework correctness, injector lifecycle integrity, adapter behavior, deviation metric accuracy, and report serialization fidelity. It does not constitute an empirical fault-injection study; physical device experiments are planned as a companion contribution [9].

4.1 Test Suite Structure

The 246 specifications cover five system dimensions:

Injector unit tests (78 cases). Each of the 11 fault injectors is tested for: parameter boundary validation (severity clamped to $[0.0, 1.0]$); correct scaling of internal parameters (memory budget, latency multiplier, corruption intensity) as a function of severity; clean resource release after `cleanup()` with zero memory leaks verified by heap snapshot comparison; and no-op behavior at $s = 0.0$.

Adapter integration tests (52 cases). Each of the 8 model adapters is tested for: initialization state management (`isInitialized` transitions); correct input tensor shape transformation and type casting; output parsing into `ModelOutput` schema; and graceful error propagation on invalid input without uncaught exceptions. **MockAdapter** provides deterministic outputs that enable comparison across all integration tests without requiring physical model files.

Baseline regression verification (44 cases). **BaselineManager** is tested for: golden baseline storage and retrieval from SQLite; metric deviation computation accuracy (verified against manually computed reference values); correct flagging of regressions exceeding $\pm 15\%$ on ΔL , ΔC , and ΔM ; and no false positives within the tolerance band.

Quantization degradation analysis (28 cases). QuantizationAnalyzer is tested across simulated precision reductions $FP32 \rightarrow FP16 \rightarrow INT8 \rightarrow INT4$, verifying that degradation scores are monotonically non-decreasing with quantization depth and that the scoring formula produces values in $[0.0, 1.0]$.

Report and database verification (44 cases). JsonReport, ReportDatabase, and the HTML dashboard template are tested for: JSON schema validity; SQLite persistence and retrieval round-trips; and Chart.js dashboard template rendering with correct metric binding.

4.2 Verification Results

Table 1 summarizes the test suite execution results. All 246 specifications pass on the reference platform (Dart SDK 3.4, Flutter 3.22, macOS and Linux CI environments via GitHub Actions).

Table 1: SATE AI verification test suite results ($N = 246$ specifications).

Module / Component	Test Cases	Execution Scope	Verification Target	Status
Fault Injectors (11 types)	78	Input/output mutation & OS stress	Severity bounds & memory release	Pass (100%)
Model Adapters (8 types)	52	Engine invocation & tensor mapping	Initialization & error handling	Pass (100%)
Baseline & Deviation Engine	44	Statistical threshold comparison	Regression detection accuracy	Pass (100%)
Quantization Analyzer	28	Precision scaling evaluation	Accuracy & latency trade-off scoring	Pass (100%)
Report & Database Storage	44	Serialization & HTML rendering	Schema integrity & dashboard export	Pass (100%)
Total	246			Pass (100%)

Zero memory leaks were detected during injector lifecycle testing. Metric deviation computations matched reference values to within floating-point precision ($< 10^{-9}$ relative error). The MockAdapter produces identical outputs across all integration tests, confirming that deviation flags are triggered by injector behavior rather than adapter non-determinism.

5 Discussion

5.1 Practical Applications in Mobile AI Development

SATE AI addresses gaps in the mobile ML development lifecycle at three distinct stages.

Pre-release reliability CI/CD. Integrating SATE AI into an automated test pipeline allows teams to run fault scenarios on every pull request. A MemoryPressureInjector run at $s = 0.8$

(approximately 1.2 GB allocation) can surface memory management bugs that only appear when the OS begins paging, before these bugs reach users.

Quantization precision trade-off analysis. The `QuantizationAnalyzer` and `QuantizationDriftInjector` allow teams to measure the actual confidence degradation when compressing a FP32 reference model to 4-bit GGUF or INT8 TFLite formats [2,7], rather than relying solely on benchmark dataset accuracy numbers that may not reflect the application’s production input distribution.

Fallback logic verification. Application UIs that display fallback states when model performance degrades (reduced confidence warnings, loading indicators, graceful error screens) can be tested systematically by configuring `ThermalThrottleInjector` or `MemoryPressureInjector` at increasing severity levels and verifying that the correct UI state appears at each threshold.

5.2 Relationship to the EdgePulse Observability Study

SATE AI and EdgePulse [9] are complementary tools designed to be used in sequence. SATE AI injects fault conditions into the application environment; EdgePulse captures the resulting runtime behavior—memory, thermal state, battery draw, CPU utilization, and per-inference latency—as structured `InferenceTrace` records. The combination constitutes a closed-loop reliability testing pipeline: inject a fault, observe the device’s response, and measure whether the application handles the degradation gracefully.

The planned companion study [9] will report the first empirical results from this pipeline, applying SATE AI’s injectors to real models on physical Android hardware and measuring each fault’s effect through EdgePulse telemetry.

6 Limitations

Application-boundary scope. SATE AI operates at the software application boundary. Memory pressure and thermal throttling are simulated through controlled native allocations and execution delays rather than through kernel register modification or physical hardware stress. Some failure modes that arise from hardware-specific behavior below the OS abstraction layer—microarchitectural pipeline stalls, cache eviction patterns, DRAM refresh penalties—are not reproducible through software-level injection.

Platform GPU driver variability. The `GpuMemoryPressureInjector` relies on software buffer allocation within the application sandbox. Its behavior may vary across vendor-specific GPU driver implementations that expose different memory pool semantics under pressure.

Deterministic thermal simulation. The `ThermalThrottleInjector` introduces fixed latency multipliers rather than reproducing the true feedback dynamics of DVFS, which adjusts clock speed continuously in response to junction temperature measurements. Real thermal throttling produces variable latency patterns that may differ from the injector’s piecewise linear model.

No physical device validation in this paper. The test suite validates framework correctness and API contracts; it does not validate that SATE AI’s injectors produce physically realistic failure signatures on real hardware. That question is the subject of the planned empirical study.

7 Conclusion

Mobile on-device AI lacks the reliability testing infrastructure that cloud systems take for granted. Standard unit tests cannot replicate the memory pressure, thermal throttling, and tensor corruption that production models encounter on consumer hardware. This paper has presented SATE AI, an open-source framework that brings software-level fault injection to the on-device ML runtime boundary.

The framework provides 11 fault injectors covering the principal hardware failure vectors, 8 model runtime adapters covering the principal mobile inference engines, and a baseline management subsystem that records golden performance profiles, computes metric deviations (ΔL , ΔC , ΔM), and generates structured regression reports. A 246-case automated test suite confirms that all injector lifecycle contracts, adapter behaviors, deviation computations, and report serialization paths function correctly across the reference platform.

SATE AI is available at https://pub.dev/packages/sate_ai and https://github.com/assassinaj602/sate_ai under the MIT license. An empirical follow-up study applying these injectors to physical Android hardware and measuring runtime degradation with EdgePulse telemetry [9] is in preparation.

8 References

- 1 Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., and Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, pp. 265–283. USENIX. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- 2 Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '18)*, pp. 2704–2713. <https://doi.org/10.1109/CVPR.2018.00286>
- 3 Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of the 11th International Conference on Learning Representations (ICLR '23)*. <https://arxiv.org/abs/2210.17323>
- 4 Lee, J., Kang, D., and Kim, D. (2019). On-device neural network execution for mobile AI applications. *IEEE Circuits and Systems Magazine*, 19(2), 24–39. <https://doi.org/10.1109/MCAS.2019.2909033>
- 5 Dutta, S., Gupta, S., Chaudhuri, P., and Dasgupta, P. (2021). MicroFI: A non-intrusive fault injection framework for microprocessor functional verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(6), 1102–1115. <https://doi.org/10.1109/TCAD.2020.3013000>
- 6 David, R., Duke, J., Jain, A., Janapa Reddi, V., Jeffries, N., Li, J., Kreeger, N., Nappier, I., Natraj, M., Wang, T., Warden, P., and Rhodes, R. (2021). TensorFlow Lite Micro: Embedded machine learning on TinyML systems. In *Proceedings of Machine Learning and Systems (MLSys '21)*, vol. 3, pp. 800–811. https://proceedings.mlsys.org/paper_files/paper/2021/hash/

- 7 Han, S., Mao, H., and Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *Proceedings of the 4th International Conference on Learning Representations (ICLR '16)*. <https://arxiv.org/abs/1510.00149>
- 8 Wang, X., Chen, Y., Wu, X., and Li, T. (2024). Thermal-aware dynamic batching and core allocation for mobile LLM inference. *ACM Transactions on Embedded Computing Systems*, 23(4), 1–22. <https://doi.org/10.1145/3649871>
- 9 Ullah, M. A. (2026). EdgePulse: A runtime observability framework for quantized large language models on consumer edge devices. *Zenodo*. <https://doi.org/10.5281/zenodo.23248718>